

COMP20008 Elements of Data Processing



Lecture 7: Document Processing (“document representations”)

Semester 2 2026

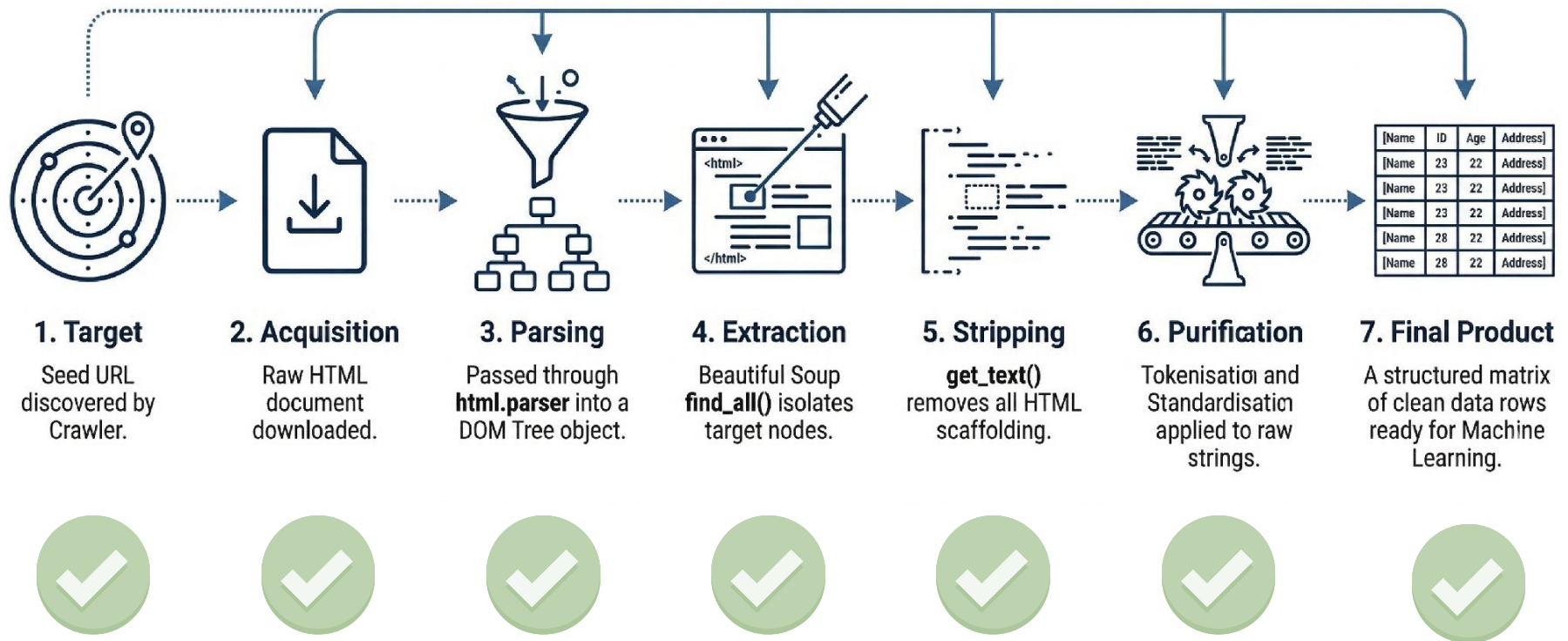
Subject Coordinator: **Dr Ekaterina Vylomova**

Co-lecturer: **Dr Hanzhuo (Vivian) Ma**



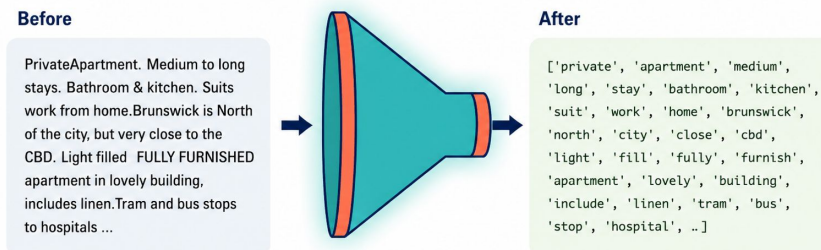
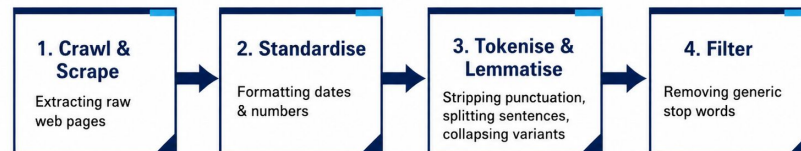
THE UNIVERSITY OF
MELBOURNE

The Complete Architecture



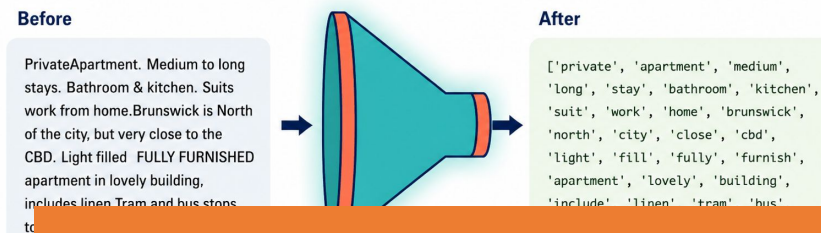
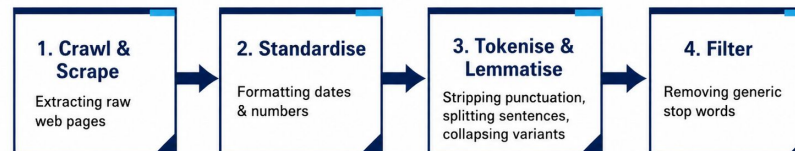
The pipeline here so far

1. We **crawled pages** from the web
2. We **scraped items** from each webpage
3. We **standardised the structured** (numerical, date, etc.) data
4. For the **‘unstructured’ text data**, we split the sentences, did case folding, tokenised and lemmatised the words, removed stop words, and did some other cleanup...
5. So we ended up with:



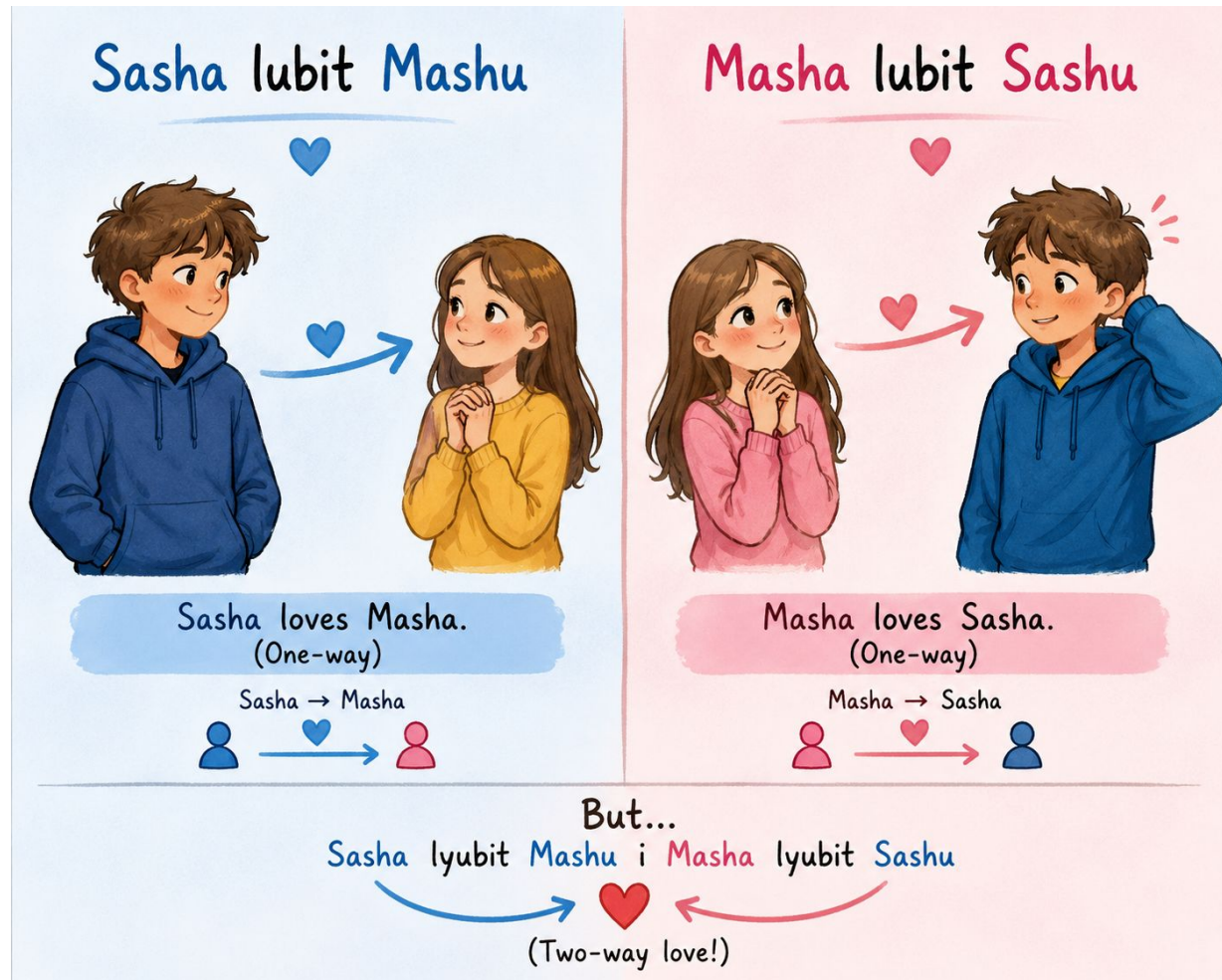
The pipeline here so far

1. We **crawled pages** from the web
2. We **scraped items** from each webpage
3. We **standardised the structured** (numerical, date, etc.) data
4. For the **‘unstructured’ text data**, we split the sentences, did case folding, tokenised and lemmatised the words, removed stop words, and did some other cleanup...
5. So we ended up with:

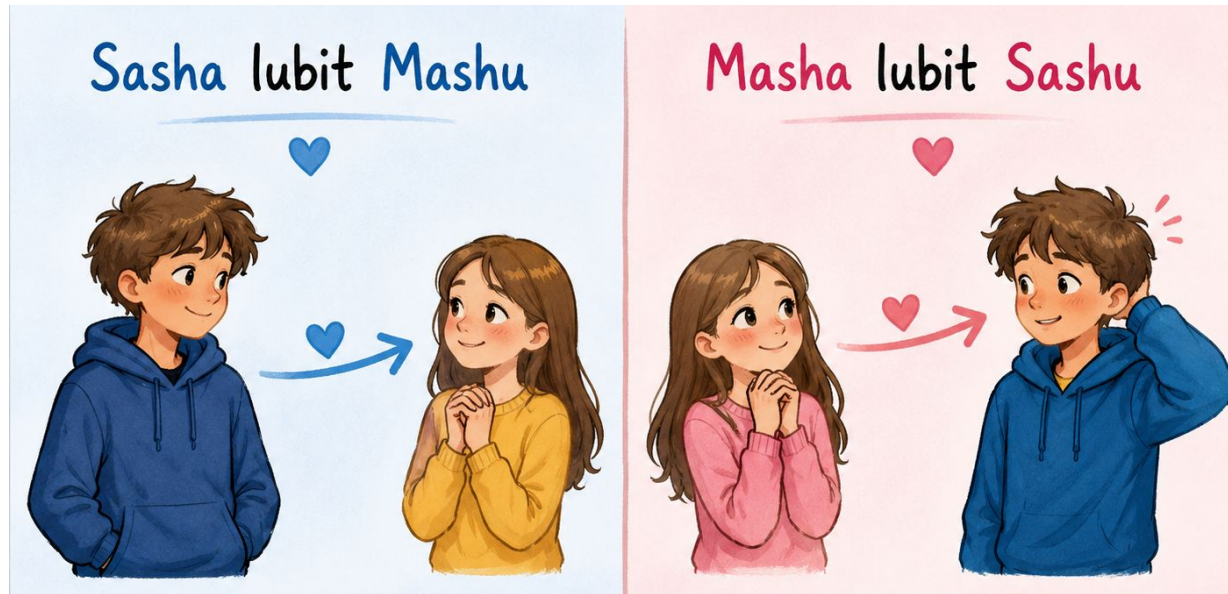


losing language structure!

Language IS Structured



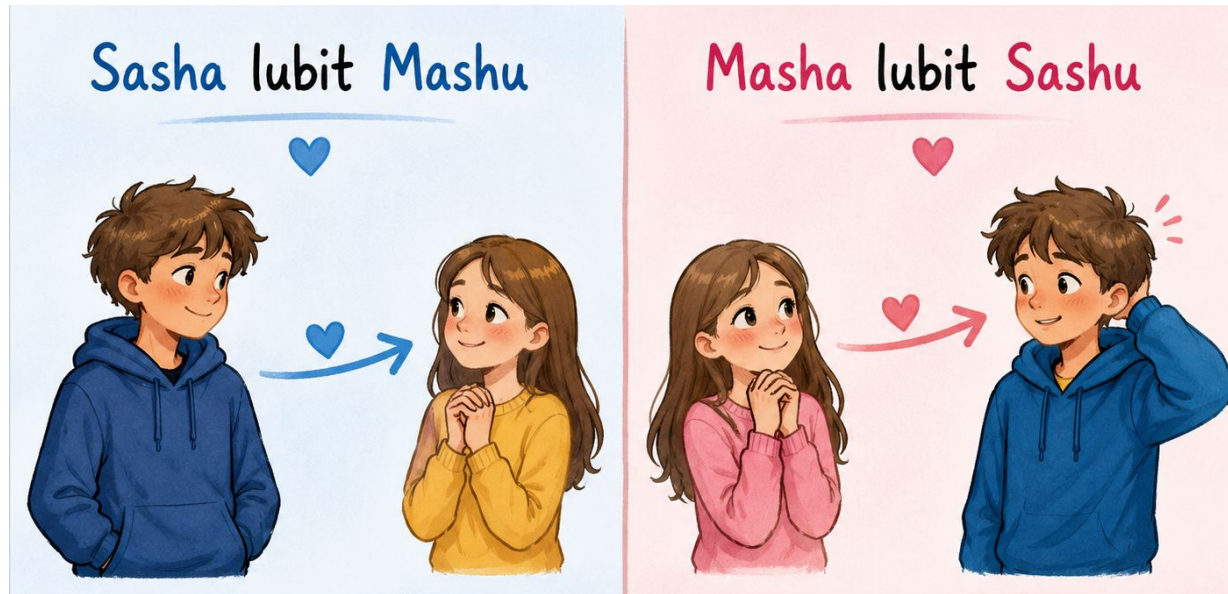
Language IS Structured



“Who does what”:

- English: word order (Subject-Verb-Object)
- Russian: Morphology (grammatical case endings); word order is flexible

Language IS Structured



“Who does what”:

- English: word order (Subject-Verb-Object)
- Russian: Morphology (grammatical case endings); word order is

In Ru: This info is lost after lemmatisation

Language IS highly ambiguous

"I saw a man with a telescope."

—— One sentence. Two interpretations. ——

1 I saw a man through a telescope.
(I used the telescope to see a man.)



2 I saw a man who had a telescope.
(The man I saw had a telescope.)



Why ambiguous?

The phrase "with a telescope" can attach in different ways in English.

Context is key! The real meaning depends on the situation.

Words are Polysemous

Eat. Think. Move at Melbourne Connect is designed to promote wellness and a healthy lifestyle in our community. Featuring fitness classes, cooking workshops, and informative sessions, this program offers you the chance to connect with your community and **start the year on the right foot**.

on the right foot



start the year on the right foot



begin the year in a positive, successful, or well-organized way

Language has nested Constructions

[*Eat. Think. Move*] at [Melbourne Connect] is **designed to promote [wellness and a healthy lifestyle] in our community**. [Featuring [fitness classes], [cooking workshops], and [informative sessions]], this program [offers you [the chance to connect with your community and start the year [on the right foot]]].

- There are units, usually in groupings (clauses)
 - The units carry information by themselves
 - The groupings also carry information
 - But no-one gives you the groupings or roles!
- With unstructured data
you have to determine the groupings and their roles yourself!

Language has nested Constructions

[*Eat. Think. Move*] at [Melbourne Connect] is **designed to promote [wellness and a healthy lifestyle] in our community**. [Featuring [fitness classes], [cooking workshops], and [informative sessions]], this program [offers you [the chance to connect with your community and start the year [on the right foot]]].

- There are units, usually in groupings (clauses)

- The units carry information by
- The groupings also carry information
- But no-one gives you the groupings

We need to make some simplifying assumptions!

- With unstructured data
you have to determine the groupings and their roles yourself!

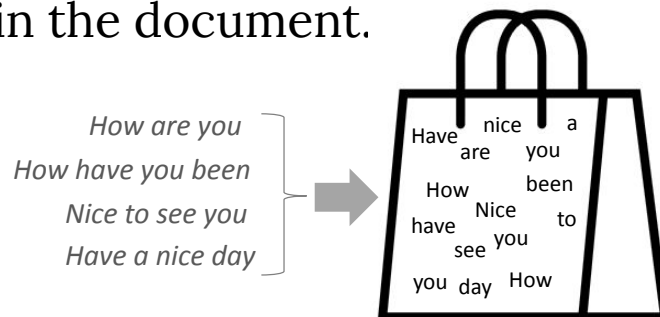
BoW: Words as features

For certain tasks such as text classification (e.g., spam vs not spam), information retrieval, or topic modelling we are primarily interested in the words that appear there and can disregard word order and grammar.

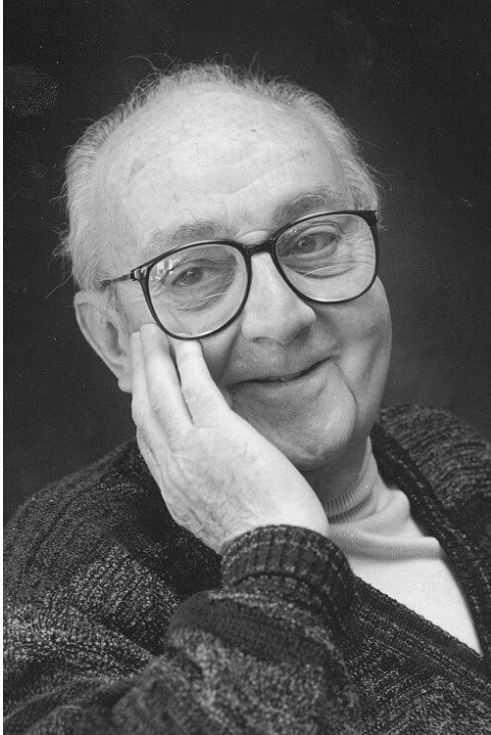
A **bag-of-words model (BoW)** is a representation of text that describes **the occurrence of words** within a text (document). It involves two things:

- A vocabulary of known words
- A measure of the presence of known words.

It is called a “**bag**” of words, because any information about the **order or structure of words in the document is discarded**. The model is only concerned with whether known words occur in the document, not where in the document.

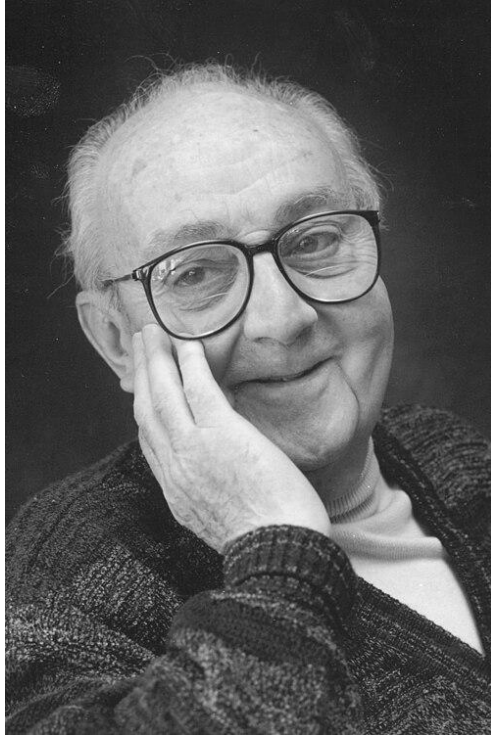


George Box' time



"All models are wrong,
but some are useful"

George Box' time



"All models are wrong,
but some are useful"

BoW allows us to evaluate similarities between texts (documents), thus enabling to cluster, classify, and rank them

Bag of Words: Web Search (IR)

The screenshot shows a Google search interface with the query 'computer science'. The results list 10 items, including links to Stanford University, MIT, ACM, Carnegie Mellon University, Wikipedia, University of California Berkeley, Computing Research Association, Usenet Newsgroups, IEEE Computer Society, and Computer Science Resources. On the right, there are sidebars for 'Tips for better searching', 'Try Google searches on:', and 'About Google!'.

Google! BETA

Search the web using Google!

computer science Google Search I'm feeling lucky

Search results for **computer science** Results 1 - 10 of about 234,000 for [computer science](#). (0.23 seconds)

1. [Computer Science Department - Stanford University](#)
Information on undergraduate and graduate programs, faculty, research, and resources.
[www.stanford.edu/dept/cs/](#) - 8k - [Cached](#) - [Similar pages](#)
2. [MIT Department of Electrical Engineering and Computer Science](#)
Academic programs, research areas, faculty, and news.
[web.mit.edu/eecs/](#) - 6k - [Cached](#) - [Similar pages](#)
3. [ACM :: Association for Computing Machinery](#)
Professional organization for computing professionals. Publications, conferences, and membership information.
[www.acm.org/](#) - 9k - [Cached](#) - [Similar pages](#)
4. [Carnegie Mellon University - School of Computer Science](#)
Overview of the school, people, research, and educational programs.
[www.cs.cmu.edu/](#) - 7k - [Cached](#) - [Similar pages](#)
5. [Computer Science - Wikipedia](#)
Overview of computer science, including links to topics, institutions, and history.
[wikipedia.org/wiki/Computer_science](#) - 25k - [Cached](#) - [Similar pages](#)
6. [University of California, Berkeley - Computer Science Division](#)
Information about the division, research, news, and academic programs.
[www.cs.berkeley.edu/](#) - 10k - [Cached](#) - [Similar pages](#)
7. [Computing Research Association](#)
Information about computing research and graduate study.
[cra.org/](#) - 6k - [Cached](#) - [Similar pages](#)
8. [Usenet Newsgroup: comp.lang.c](#)
Discussion group for the C programming language.
[groups.google.com/groups?selm=comp.lang.c](#) - 5k - [Cached](#) - [Similar pages](#)
9. [IEEE Computer Society](#)
Resources, publications, conferences, and membership information.
[www.computer.org/](#) - 7k - [Cached](#) - [Similar pages](#)
10. [Computer Science Resources](#)
Collection of links to computer science departments, labs, and online resources.
[www.csresources.org/](#) - 4k - [Cached](#) - [Similar pages](#)

Tips for better searching:

- Use specific keywords.
- Try different keywords.
- Use quotes for exact phrases.

Try Google searches on:

[Stanford](#)
[Linux](#)
["Artificial Intelligence"](#)
["Neural Networks"](#)

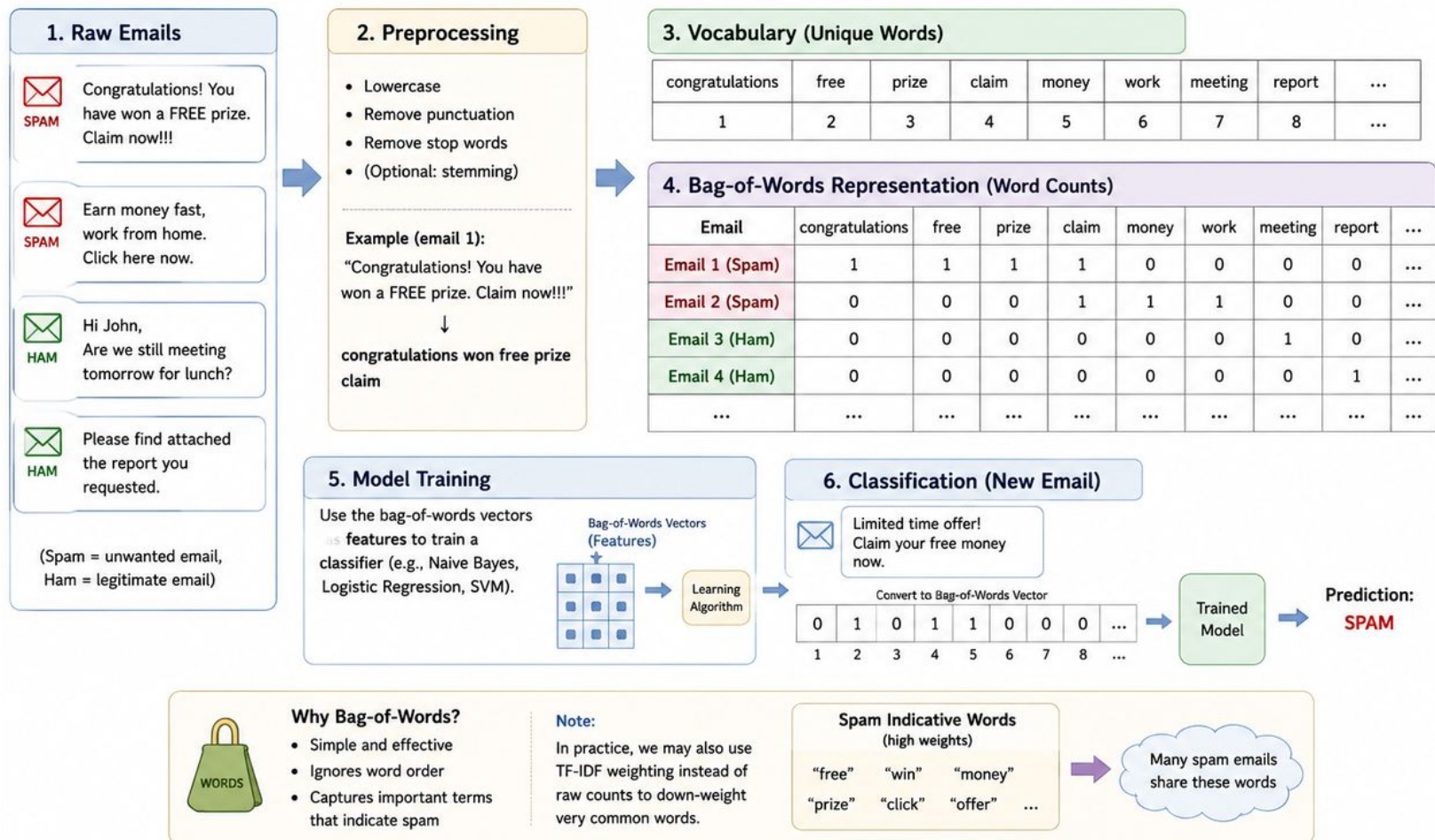
About Google!

[Stanford Search](#) [Linux Search](#)
[About Google!](#) [Google! Logos](#)

Copyright ©1998 [Google Inc.](#)

- Document: a single web page
- Used to evaluate a relevance score for each page to user query
- rank documents by relevance

Bag of Words: Spam Detection



Each email is a separate document

Bag of Words (BoW)



A vocabulary of known words

a	are	been	day	have	how	nice	see	to	you
---	-----	------	-----	------	-----	------	-----	----	-----

* WO = occurrence of words

[WO_a, WO_{are}, WO_{been}, WO_{day}, WO_{have}, WO_{how}, WO_{nice}, WO_{see}, WO_{to}, WO_{you}]

How are you = [0, 1, 0, 0, 0, 1, 0, 0, 0, 1]

How have you been = [0, 0, 1, 0, 1, 1, 0, 0, 0, 1]

Nice to see you = [0, 0, 0, 0, 0, 0, 1, 1, 1, 1]

Have a nice day = [1, 0, 0, 1, 1, 0, 1, 0, 0, 0]

Total Frequency = [1, 1, 1, 1, 2, 2, 2, 1, 1, 3]

a	are	been	day	have	how	nice	see	to	you
1	1	1	1	2	2	2	1	1	3

BoW representation for text

The simplest representation of a document

- Each document is a **statistical distribution**: a count of every word it contains
- This info is captured in a **numeric vector**
 - Each cell (each *dimension/feature*) is a specific word in the *lexicon*
 - The value in each cell is the number of times (the *count*) of that word in the document

Total Frequency = [1, 1, 1, 1, 2, 2, 2, 1, 1, 3]

a	are	been	day	have	how	nice	see	to	you
1	1	1	1	2	2	2	1	1	3

BoW representation for text

The simplest representation of a document

- Each document is a **statistical distribution**: a count of every word it contains
- This info is captured in a **numeric vector**
 - Each cell (each *dimension/feature*) is a specific word in the *lexicon*
 - The value in each cell (the *count*) of that word in

We need to reduce the number of features (data sparsity)

Total Frequency = [1, 1, 1, 1, 2, 2, 2, 1, 1, 3]

a	are	been	day	have	how	nice	see	to	you
1	1	1	1	2	2	2	1	1	3

Example from AirBnB

\$571 An extraordinary St Kilda mansion

Exterior walls feature crumbling pink paint, with an orange grove to greet you in the front garden. Beautifully and simply restored it has a French sensibility inside and an expansive garden. 4m high ceilings downstairs and upstairs. Rich with art and unique furnishings it has a glorious ambience inside

\$52 Melbourne, Brunswick, Aptmnt, CityViews

Private Apartment. Medium to long stays. Bathroom & kitchen. Suits work from home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED apartment in lovely building, includes linen. Tram and bus stops to hospitals and universities at front of building. Nearby: grocery, pharmacy, wine, nuts, bakery, restaurants, and take-aways. 200gb data refresh monthly on 27th then Guest pay for own WIFI

???? Californian Bungalow

Charming and hugely spacious, the house offers a unique experience and so close to the City. The house itself is set in a quiet suburb near the bike path to the City. It has its own beautiful pool and outdoor area. It consists of six full bedrooms and two bathrooms. The attention to detail of the finishing throughout the house and the high ceilings makes the whole space feel very relaxed. Three bedrooms downstairs and three upstairs. One full bathroom downstairs and upstairs.

Example from AirBnB

doc1 = “Exterior walls feature ...”

doc2 = “PrivateApartment. Medium to long stays...”

doc3 = “Charming and hugely spacious, ...”

Preprocessing:

Vectoriser:

```
def preprocess(text):  
    doc = nlp(text.lower())  
  
    tokens = [  
        token.lemma_  
        for token in doc  
        if not token.is_punct  
        and not token.is_space  
    ]
```

```
    return " ".join(tokens)
```

```
processed_docs = [preprocess(doc)  
    for doc in documents]
```

```
from sklearn.feature_extraction.text import  
CountVectorizer  
  
vectorizer = CountVectorizer()  
  
bow = vectorizer.fit_transform(processed_docs)
```

3x128

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs	garden	house	inside
doc1	1	0	1	0	0	0	1	0	1	2	0	2
doc2	0	2	0	1	0	2	0	1	0	0	0	0
doc3	0	0	0	2	2	0	1	2	2	0	3	0

Example from AirBnb

doc1 = “Exterior walls feature ...”

doc2 = “PrivateApartment. Medium to long stays...”

doc3 = “Charming and hugely spacious, ...”

Preprocessing:

Vectoriser:

```
def preprocess(text):  
    doc = nlp(text.lower())  
  
    tokens = [  
        token.lemma_  
        for token in doc  
        if not token.is_punct  
        and not token.is_space  
    ]
```

```
    return " ".join(tokens)
```

```
processed_docs = [preprocess(doc)  
    for doc in documents]
```

```
from sklearn.feature_extraction.text import  
CountVectorizer
```

```
vectorizer = CountVectorizer()
```

```
bow = vectorizer.fit_transform(processed_docs)
```

The matrix is often huge and sparse!

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs	garden	house	inside
doc1	1	0	1	0	0	0	1	0	1	2	0	2
doc2	0	2	0	1	0	2	0	1	0	0	0	0
doc3	0	0	0	2	2	0	1	2	2	0	3	0

Example from AirBnb

doc1 = “Exterior walls feature ...”

doc2 = “PrivateApartment. Medium to long stays...”

doc3 = “Charming and hugely spacious, ...”

Preprocessing:

Vectoriser:

```
def preprocess(text):  
    doc = nlp(text.lower())  
  
    tokens = [  
        token.lemma_  
        for token in doc  
        if not token.is_punct  
        and not token.is_space  
    ]
```

```
    return " ".join(tokens)
```

```
processed_docs = [preprocess(doc)  
    for doc in documents]
```

```
from sklearn.feature_extraction.text import  
CountVectorizer
```

```
vectorizer = CountVectorizer()
```

```
bow = vectorizer.fit_transform(processed_docs)
```

With more preprocessing, we can reduce the number of unique features (dimensions) in the vector.

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs	garden	house	inside
doc1	1	0	1	0	0	0	1	0	1	2	0	2
doc2	0	2	0	1	0	2	0	1	0	0	0	0
doc3	0	0	0	2	2	0	1	2	2	0	3	0

Which doc is most similar to doc3?

doc1 = “Exterior walls feature ...”

doc2 = “PrivateApartment. Medium to long stays...”

doc3 = “Charming and hugely spacious, ...”

Computing cosine similarities between 128-dimensional vectors:

```
from sklearn.metrics.pairwise import cosine_similarity

sim_doc3_doc1 = cosine_similarity(bow[2], bow[0])[0, 0]
sim_doc3_doc2 = cosine_similarity(bow[2], bow[1])[0, 0]
```

doc3 vs doc1: 0.41

doc3 vs doc2: 0.43

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs	garden	house	inside
doc1	1	0	1	0	0	0	1	0	1	2	0	2
doc2	0	2	0	1	0	2	0	1	0	0	0	0
doc3	0	0	0	2	2	0	1	2	2	0	3	0

Which doc is most similar to doc3?

doc1 = “Exterior walls feature ...”

doc2 = “PrivateApartment. Medium to long stays...”

doc3 = “Charming and hugely spacious, ...”

Computing cosine similarities between 128-dimensional vectors:

```
from sklearn.metrics.pairwise import cosine_similarity

sim_doc3_doc1 = cosine_similarity(bow[2], bow[0])[0, 0]
sim_doc3_doc2 = cosine_similarity(bow[2], bow[1])[0, 0]
```

doc3 vs doc1: 0.41

doc3 vs doc2: 0.43 Huh?

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs	garden	house	inside
doc1	1	0	1	0	0	0	1	0	1	2	0	2
doc2	0	2	0	1	0	2	0	1	0	0	0	0
doc3	0	0	0	2	2	0	1	2	2	0	3	0

Good things about BoW

	change	school	degree	year	future	meat	eat	cheese	oil	...
doc001	2	4	1	4	3	0	0	0	0	...
doc002	1	3	3	1	4	0	1	0	0	...
doc003	0	2	0	0	2	3	3	4	2	...
...										

Now **we can easily compare documents:**

1. Very simple representation (just integers)
2. All we need is a very long and simple vector (how long?)
3. Easy to produce using the preprocessing tools
4. Works for any language and for emoticons, etc. too!

Problem with BoW

Discarding word order ignores the context (the meaning of words in the document = semantics).

same word counts! $\left(\begin{array}{l} S1 = I \text{ love you but you hate me} \\ S2 = I \text{ hate you but you love me} \end{array} \right.$



BoW similarity

	change	school	degree	year	future	meat	eat	cheese	oil	...
doc001	2	4	1	4	3	0	0	0	0	...
doc002	1	3	3	1	4	0	1	0	0	...
doc003	0	2	0	0	2	3	3	4	2	...
...										

Two documents with similar BoW vectors are probably talking about the same topic.

BoW uses **Term Frequency** as its values

Can you see any patterns?

BoW similarity

	change	school	degree	year	future	meat	eat	cheese	oil	...
doc001	2	4	1	4	3	0	0	0	0	...
doc002	1	3	3	1	4	0	1	0	0	...
doc003	0	2	0	STUDIES		3	3	4	2	...
...									FOOD	

Two documents with similar BoW vectors are probably talking about the same topic.

Can you see any patterns?

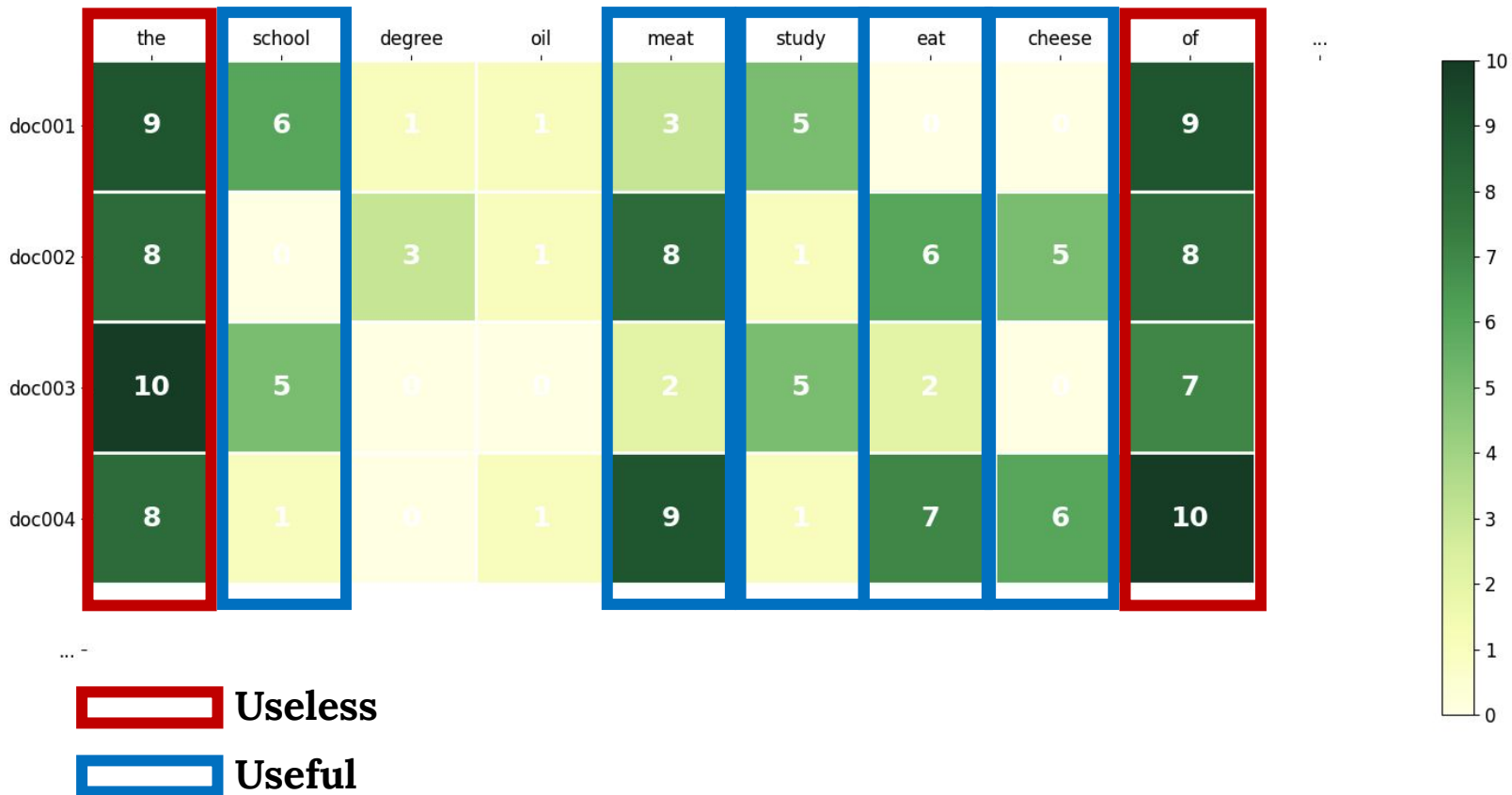
Vector Similarity

	the	school	degree	oil	meat	study	eat	cheese	of	...
doc001	9	6	1	1	3	5	0	0	9	...
doc002	8	0	3	1	8	1	6	5	8	...
doc003	10	5	0	0	2	5	2	0	7	...
doc004	8	1	0	1	9	1	7	6	10	...

- Check Term Frequency (Counting)!
- Some words are more helpful than others!
Which? The unique ones!

Can you see any patterns?

Vector Similarity



Is this enough?

Is Term Frequency enough?

Document A

“python python good
python python ...”

TF (Python) = 4

Document B

“python tutorial beginner
guide to install ...”

TF (Python) = 1

Term Frequency alone may give the wrong answer

TF says: **Document A** is more relevant to Python.

Is Term Frequency enough?

Document A

“python python good
python python ...”

TF (Python) = 4

Document B

“python tutorial beginner
guide to install ...”

TF (Python) = 1

“More counts $\neq$ more meaning”

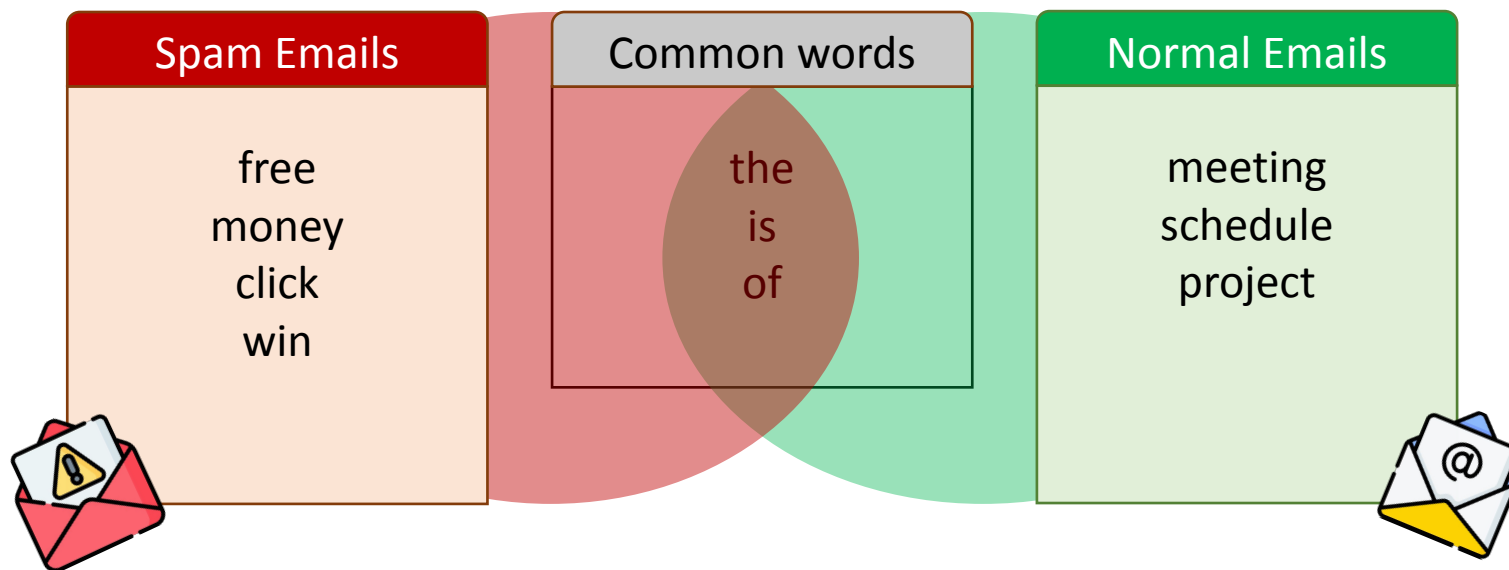
Reasons to avoid **simple counts**:

1. **Common words dominate the counts**
(Counts do not capture important (rare) words that would indicate something unusual of the document)
2. **Counts depend on document length**
(If it's long, the count usually increases.)

Rewarding word rareness

When comparing documents:

- **Common words** (*The, a, of*) appear everywhere! – **NOT useful**
- **Rare words** (appear in only a few documents) **distinguish documents.**



IDF: Rewarding word rareness

We want to reward words that appear in only a few documents: **The fewer, The better**

$$\text{reward}_w = \frac{\text{total number of documents}}{\text{number of documents containing } w}$$

The fewer documents contain the word,
the higher its importance!

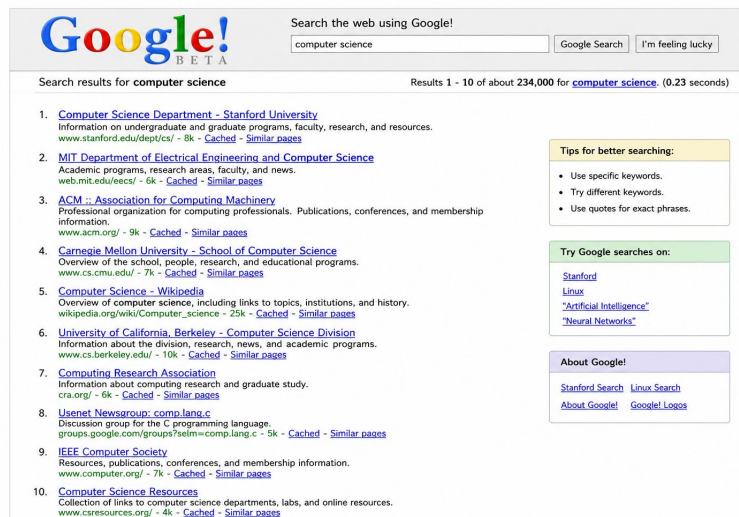
- This is called **IDF (Inverse Document Frequency)**

The **TF-IDF** model

TF × **IDF** = **word weight**

“Local Abundance”
A word is mathematically stronger if it appears frequently within a specific document.

“Global Rarity”
A word is stronger if it is highly localized, appearing in only a few documents across the entire collection.



One of the major models in Information Retrieval (Web Search)!

TF and IDF

TF = Term Frequency
IDF = Inverse Document Frequency

Important words are **frequent in one document**,
but **rare across others**.

So, You just multiply them:

$$\text{TF} \times \text{IDF} = \text{word weight}$$

TF “Local Abundance”
A word is mathematically stronger
if it appears frequently
within a specific document.

IDF “Global Rarity”
A word is stronger
if it is highly localized,
appearing in only a few documents across the entire
collection.

TF and IDF

TF = Term Frequency
IDF = Inverse Document Frequency

Important words are **frequent in one document**,
but **rare across others**.

So, You just multiply them:

a word 100x more rare does
not make a doc 100s more
relevant

$$tf_{i,j} \times \frac{N}{df_i} = w_{i,j}$$

$w_{i,j}$ = weight of term i in document j

$tf_{i,j}$ = number of occurrences of term i in document j

N = total number of documents

df_i = number of documents containing term i

TF and IDF

TF = Term Frequency
IDF = Inverse Document Frequency

Important words are **frequent in one document**, but **rare across others**.

So, You just multiply them:

$$tf_{i,j} \times \log\left(\frac{N}{df_i}\right) = w_{i,j}$$

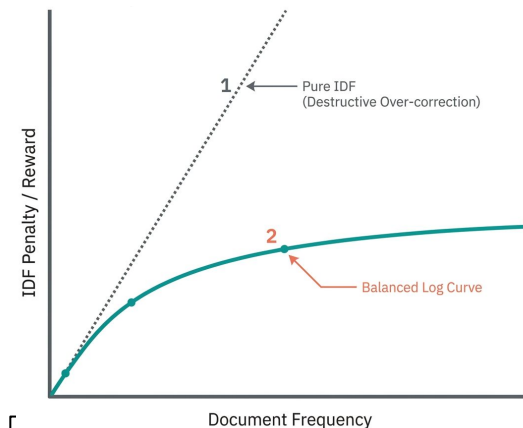
$w_{i,j}$ = weight of term i in document j

$tf_{i,j}$ = number of occurrences of term i in document j

N = total number of documents

df_i = number of documents containing term i

Solution: *log*



You can use the **log of IDF** so this makes it grow more slowly.

Is Term Frequency enough?

Document A

“python python good
python python ...”

TF (Python) = 4

Document B

“python tutorial beginner
guide to install ...”

TF (Python) = 1

“More counts $\neq$ more meaning”

Reasons to avoid **simple counts**:

1. **Common words dominate the counts**
(Counts do not capture important (rare) words that would indicate something unusual of the document)
2. **Counts depend on document length**
(If it's long, the count usually increases.)

Normalising (Rescaling) **TF-IDF**

Longer documents → larger TF-IDF values. This makes the comparison unfair.

Let's normalise TF-IDF:

- **Strategy 1:** Divide by the total length (in tokens) of the document
- **Strategy 2:** Divide the TF-IDF scores by the sum of their weights to make values $\in [0,1]$

$$tf_{i,j} \times \log\left(\frac{N}{df_i}\right) = w_{i,j}$$

$$\frac{w_{i,j}}{\sqrt{\sum_i w_{i,j}^2}} = \text{Normalised TFIDF}$$

TF-IDF Examples #1

Two documents: A – “the **car** is **driven** on the **road**”

B – “the **truck** is **driven** on the **highway**”

The terms are between 0 and 1.
Terms (local to one doc) are stronger

word	A	B
car	1	0
driven	1	1
road	1	0
truck	0	1
highway	0	1

A	B	A	B
1.405	0	0.632	0
1	1	0.449	0.449
1.405	0	0.632	0
0	1.405	0	0.632
0	1.405	0	0.632

- N is number of documents = 2
- $\ln$ is natural log

$$\sqrt{\sum_{t' \in A} v_{t'}^2} \quad \sqrt{\sum_{t' \in B} v_{t'}^2}$$

Calculating **TFIDF** for this example

```
[1]: from sklearn.feature_extraction.text import TfidfTransformer
```

```
# A 'the car is driven on the road'
```

```
# B 'the truck is driven on the highway'
```

```
# [car, driven, road, truck, highway]
```

```
term_counts = [[1, 1, 1, 0, 0],  
               [0, 1, 0, 1, 1]]
```

```
transformer = TfidfTransformer() #  $idf(t) = \log\left(\frac{1+N}{1+df(t)}\right) + 1$   
tfidf = transformer.fit_transform(term_counts)
```

```
[2]: transformer.idf_
```

```
[2]: array([1.40546511, 1.         , 1.40546511, 1.40546511, 1.40546511])
```

```
[3]: tfidf.toarray()
```

```
[3]: array([[0.6316672 , 0.44943642, 0.6316672 , 0.         , 0.         ], Doc A  
          [0.         , 0.44943642, 0.         , 0.6316672 , 0.6316672 ]]) Doc B
```


Using **TF-IDF** instead of Frequency

Replace raw frequency counts with **TF-IDF** scores

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs	garden	house	inside
doc1	1	0	1	0	0	0	1	0	1	2	0	2
doc2	0	2	0	1	0	2	0	1	0	0	0	0
doc3	0	0	0	2	2	0	1	2	2	0	3	0

Now the text document has a new layer of structure (that reflects uniqueness to a document)

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs
doc1	0.1377	0.0000	0.1377	0.0000	0.0000	0.0000	0.1047	0.0000	0.1047
doc2	0.0000	0.2419	0.0000	0.0920	0.0000	0.2419	0.0000	0.0920	0.0000
doc3	0.0000	0.0000	0.0000	0.1355	0.1782	0.0000	0.0678	0.1355	0.1355

Using **TF-IDF** instead of Frequency

Replace raw frequency counts with **TF-IDF** scores

	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs	garden	house	inside
doc1	1	0	1	0	0	0	1	0	1	2	0	2
doc2	0	2	0	1	0	2	0	1	0	0	0	0
doc3	0	0	0	2	2	0	1	2	2	0	3	0

Now the text document has a new layer of structure (that reflects uniqueness to a document)

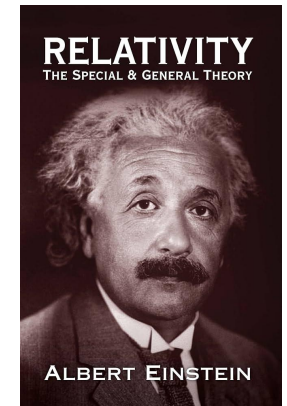
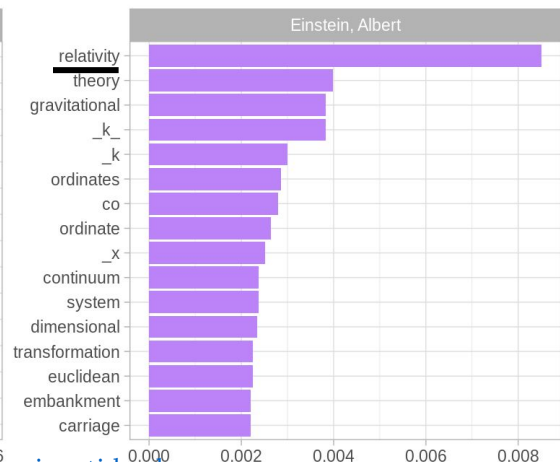
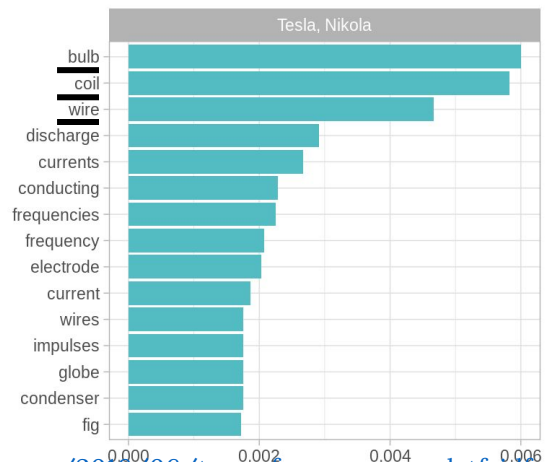
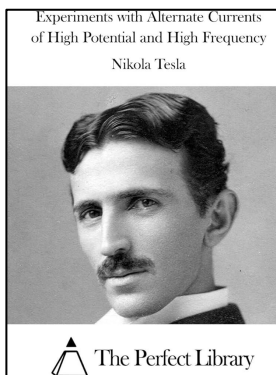
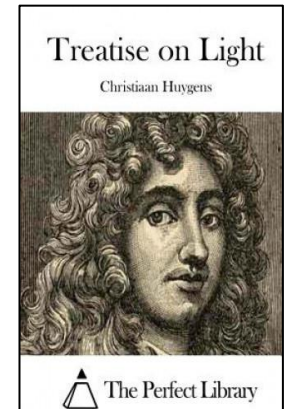
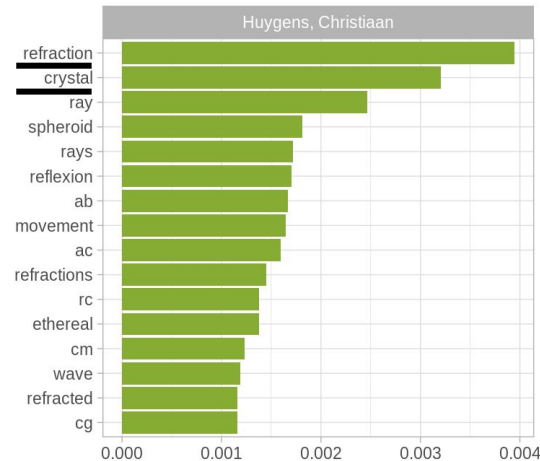
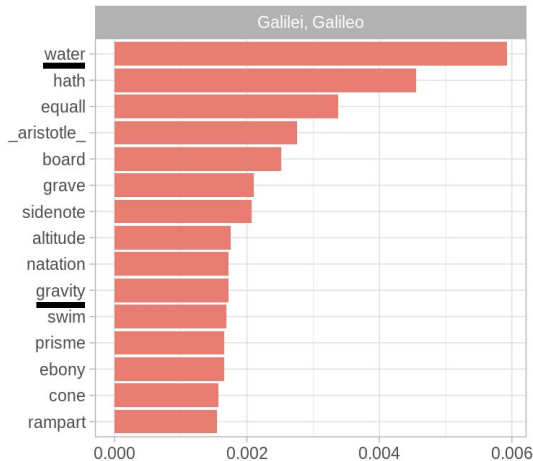
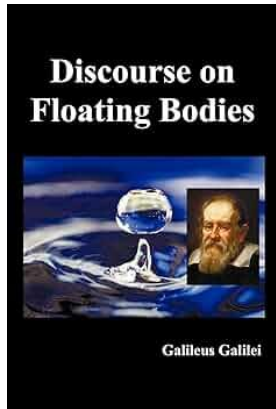
	ambience	apartment	art	bathroom	bedroom	building	ceiling	city	downstairs
doc1	0.1377	0.0000	0.1377	0.0000	0.0000	0.0000	0.1047	0.0000	0.1047
doc2	0.0000	0.2419	0.0000	0.0920	0.0000	0.2419	0.0000	0.0920	0.0000
doc3	0.0000	0.0000	0.0000	0.1355	0.1782	0.0000	0.0678	0.1355	0.1355

doc3 vs doc1: 0.27

doc3 vs doc2: 0.26

TF-IDF Examples #2

These are NOT JUST frequent words; they are **distinctive** to each document.



<https://www.r-bloggers.com/2016/06/term-frequency-and-tf-idf-using-tidy-data-principles/>